Prompt for Antigravity: build the StratumRO Brain (shared memory vault)

How to use: fill the three paths in the first block, then paste everything below the line into Antigravity (agent mode, with the terminal allowed). The kit `stratumro-brain-starter.zip` must already be on your computer.

Paths (owner fills these in before pasting)

- KIT_ZIP = C:\path\to\stratumro-brain-starter.zip
- VAULT_PARENT = C:\path\to\the\folder\where\the\vault\folder\will\be\created
- CODE_REPO = C:\path\to\your\StratumRO-QGIS clone

Your job

Prepare a Git-backed Obsidian vault named `stratumro-brain` that becomes the shared memory for every AI agent and for me on the StratumRO project. It must (1) be a normal Git repository of Markdown files, (2) open in Obsidian with a clean, good-looking setup, (3) let me browse the StratumRO-QGIS code repository from inside Obsidian through generated pages, and (4) be readable by Antigravity and OpenCode through `AGENTS.md` and `.agents/skills/`.

Ground rules

1. Do not invent facts, file formats, plugin ids, settings keys or command options. If something is not in the kit, in the documents linked below, or in the output of a command you ran, say "not verified" and ask me.
2. Work in small steps. After each phase print: what you did, the exact commands you ran, the result, and what failed. Stop at the checkpoints marked CHECKPOINT and wait for my answer.
3. Do not modify the code repository (CODE_REPO) except in Phase 6, on a separate branch, and only after I approve the diff.
4. Never run `git push --force`, never rewrite history, never delete notes.
5. Never put credentials, tokens, client data or survey data in the vault. If you meet one, stop and tell me.
6. Do not read the source of `scripts/*.py`; run them and use `--help`.
7. I use Windows 10/11. Use PowerShell syntax. Use `py` if `python` is not found.
8. Write for a reader who is not a programmer: short sentences, plain words.

Documents you may rely on (read them before the phase that needs them):

- Obsidian accepted file formats: https://obsidian.md/help/file-formats

- Obsidian settings: https://obsidian.md/help/settings
- Obsidian CSS snippets: https://obsidian.md/help/snippets
- Obsidian Bases syntax: https://obsidian.md/help/bases/syntax
- Obsidian command line: https://obsidian.md/help/cli
- Antigravity skills: https://antigravity.google/docs/skills/
- Antigravity rules: https://antigravity.google/docs/rules/
- OpenCode skills: https://opencode.ai/docs/skills/
- OpenCode rules: https://opencode.ai/docs/rules/

Phase 0: Inspect (no changes)

Run and report in one table: Windows version; `git --version`; `py --version`; `gh --version` and `gh auth status` (if installed); `node --version` (if installed); whether Obsidian is installed and its version (the command line needs 1.12.7 or newer); whether CODE_REPO exists, its current branch and `git status --short | Measure-Object`; whether KIT_ZIP exists. Do not install anything. List what is missing and how I install it. CHECKPOINT: wait for me.

Phase 1: Create the vault repository

1. Extract KIT_ZIP into VAULT_PARENT. The result must be `VAULT_PARENT\stratumro-brain\` with `AGENTS.md`, `INDEX.md`, `canon\`, `inbox\`, `.agents\skills\` and `scripts\` inside. If the zip has another top folder name, rename it.
2. Run `py scripts\lint_vault.py` inside it. It must report 0 errors. If not, stop and show the output.
3. `git init -b main`, `git add -A`, commit with the message `init: stratumro-brain starter vault`.
4. Ask me for the GitHub account and confirm: private repository named `stratumro-brain`. CHECKPOINT: wait for my yes. Then, if `gh` is authenticated, create it with `gh repo create stratumro-brain --private --source . --push`. If not, give me the exact steps and stop.

Phase 2: Skills for agents

The kit already contains five skills in `.agents\skills\`: stratumro-memory-write, stratumro-memory-review, stratumro-vault-lint, stratumro-code-map, stratumro-obsidian-look. Open each `SKILL.md` and confirm it has `name` equal to the folder name and a `description`.

Add the Obsidian skills from https://github.com/kepano/obsidian-skills (MIT licence):

1. `git clone --depth 1 https://github.com/kepano/obsidian-skills.git $env:TEMP\obsidian-skills`
2. Read each of `obsidian-markdown`, `obsidian-bases`, `json-canvas`, `obsidian-cli` (their SKILL.md and any scripts). Tell me if any of them runs a download, a network call or a command that is not about Obsidian files. Do not copy a skill you are unsure about.

3. Copy only those four folders into `.agents\skills\`. Copy the repository's LICENSE next to them as `.agents\skills\LICENSE-obsidian-skills.txt`.
4. Create `.agents\skills\SOURCES.md` listing each external skill, the source URL, the commit id (`git -C $env:TEMP\obsidian-skills rev-parse HEAD`) and the date.
5. Do not copy `defuddle` or `knap`.
6. Check that Antigravity lists all nine skills (Customizations menu in the agent panel). Report the result.
7. Run the linter, then commit: `skills: add obsidian skills`.

Phase 3: Obsidian setup

Steps I do myself (tell me exactly, then wait): open Obsidian, Open folder as vault, choose `VAULT_PARENT\stratumro-brain`; Settings, Community plugins, turn off Restricted mode; Settings, General, enable Command line interface and register it; leave Obsidian running.

CHECKPOINT: ask me to confirm Obsidian is open on the vault.

Then you do:

1. Run `obsidian` with `--help` or the documented list command to confirm the command line works. If it does not, use the manual route below.
2. Theme: `obsidian theme:install name="Minimal" enable` (use the exact syntax from https://obsidian.md/help/cli).
3. Plugins, approved list only: "Minimal Theme Settings" (id `obsidian-minimal-settings`), "Style Settings", "Git" by Vinzent03 (repository https://github.com/Vinzent03/obsidian-git). Verify each id in the plugin catalogue or in the plugin's `manifest.json` before installing. Install with `obsidian plugin:install id=<id> enable`.
4. Manual route if the command line fails: download `main.js`, `manifest.json` and `styles.css` from the plugin's official GitHub release into `.obsidian\plugins\<id>\` (folder name equals the id in manifest.json), then ask me to enable it in Settings, Community plugins.
5. Do not install any other plugin. If you think another one would help, propose it with the reason and wait.
6. Confirm `.obsidian\appearance.json` contains `"enabledCssSnippets": ["stratumro"]` and the theme is Minimal, and `.obsidian\app.json` contains `"showUnsupportedFiles": true`. Read the files Obsidian wrote; do not guess other keys.
7. Git plugin settings (I click, you give me the checklist): pull on startup on; auto pull every 10 minutes; auto commit-and-sync after 10 minutes without edits; push after commit on. Authentication: use the Git credentials already set up on this computer.
8. Commit: `obsidian: theme, plugins, settings`.

Phase 4: Make it look great

Design brief: calm, high contrast, dark mode first, readable at 125% Windows scaling. One colour per note type (decision green, fact blue, lesson amber, handoff purple, model note teal). A home page with dashboards.

1. Use the skill `stratumro-obsidian-look`. The kit already has `.obsidian\snippets\stratumro.css`, `HOME.md` and four `.base` files in `dashboards\`.
2. Ask me to open `HOME.md`. Check with me that: the three dashboards (Decisions, Waiting for review, Needs a re-check) and the Facts table render; decision notes have a green bar next to the title; the file list shows `canon` green and `inbox` amber.
3. If a Base shows an error, fix the `.base` file using the `obsidian-bases` skill and the syntax page. Common risk: the date comparison on `recheck_by`; if it fails, change the property type to Date in Obsidian (Properties view) and retry.
4. In Settings, Appearance, set the Minimal colour scheme through Minimal Theme Settings; propose two options and let me choose. Do not change the CSS palette unless I ask.
5. Ask me for a screenshot of `HOME.md` and of one open note. Fix only what the screenshot shows is wrong.
6. Commit: `look: theme, snippet, dashboards`.

Phase 5: Browse the code repository from Obsidian

1. Run `py scripts\code_map.py --repo CODE_REPO --out code-map`. Report the output line.
2. Open `code-map\README.md` with me and check the five pages open. The pages link every tracked file to GitHub at the commit used.
3. Ask me whether I also want a local link so Obsidian can show the real files. If yes: `cmd /c mklink /J code CODE_REPO` in the vault root (the folder `code\` is in `.gitignore`). Test with me that `code\README.md` opens; if the file list becomes slow or Obsidian struggles, remove the junction (`rmdir code`) and keep only the code map pages.
4. Explain to me in five lines which file types Obsidian opens itself (Markdown, Bases, Canvas, images, audio, video, PDF) and which open in another program (Python, JSON, QGIS projects, CSV).
5. Commit: `code-map: first generation`.

Phase 6: Connect the agents (code repository)

1. In CODE_REPO create the branch `chore/memory-pointer`.
2. Append to its `AGENTS.md` only the section "Pointer for AGENTS.md" from the vault's `MEMORY_RULES.md`, with the real path of the vault. Do not change anything else. Show me `git diff`. CHECKPOINT: wait for my approval before committing.
3. Test as a proof, not by assumption: start an OpenCode session in CODE_REPO and ask it "Read INDEX.md in the stratumro-brain vault and list the approved decisions". Do the same in Antigravity. Report whether each could read the folder outside the project. If a

permission blocks it, report the exact message and the documented setting; do not guess a setting.

4. Run the five stage-1 questions from the Memory Architecture Report and score each tool: answer correct, note id cited. Only notes in `canon\` count as a correct source, so I decide first which inbox notes to promote and tell you. Questions: which phase comes before SNAP; what law is in force for PUG; what is the Cluj recall baseline; where do agents write; who may edit canon.
5. Do not install any MCP memory server. Stop after the test and give me the score.

Phase 7: Final report

Give me one table: phase, done or not, commands, files created, what I must still click. Then list everything marked not verified. Then the three most useful next actions. Do not claim success for anything you did not test.

Definition of done

1. `py scripts\lint_vault.py` reports 0 errors.
2. The vault opens in Obsidian with the Minimal theme, the dashboards render, and the home page looks like the design brief.
3. `code-map\` pages exist and open.
4. Nine skills are visible in Antigravity.
5. The private GitHub repository holds the vault, and the vault was pulled once on a second machine (I do this part; tell me the three commands).
6. Both OpenCode and Antigravity answered the five questions using notes from the vault.